# ASSIGNMENT 02

# DOCKER AND KUBERNETES

*Containerization and Deployment of a Flask Student Management System*

Name – Atharva Gavali
PRN - 2020301070040

## 1. Aim

To containerize a Flask-based Student Management System using Docker and deploy the containerized application on a local Kubernetes cluster. The application is exposed through a Kubernetes NodePort Service and accessed through a web browser.

## 2. Objectives

- Develop and run a Flask web application for managing student records.
- Prepare a Docker image with the application code and Python dependencies.
- Run the application as a Docker container and verify its port mapping.
- Create Kubernetes Deployment and Service resources using YAML manifests.
- Run two application replicas and expose the application through NodePort 30080.
- Verify the Deployment, Pods, Service, browser access, and GitHub repository.

## 3. Software and Tools Required

| Tool | Purpose |
|---|---|
| **Python 3.11** | Runtime for the Flask application. |
| **Flask 3.1.2** | Python web framework. |
| **Visual Studio Code** | Code editor and integrated terminal. |
| **Docker Desktop** | Builds and runs the container image. |
| **Kubernetes** | Orchestrates the application containers. |
| **kubectl** | Command-line interface for Kubernetes. |
| **Git and GitHub** | Version control and remote source-code hosting. |
| **Web browser** | Verifies the application endpoint. |

# 4. Application Overview

The project is a Flask-based Student Management System. The web interface contains an Add Student form with fields for Student Name, Roll Number, and Course. It also contains a Student Records table with an Action column. The Flask server listens on port 5000 and is configured to bind to 0.0.0.0 so it can accept requests arriving through the container network.

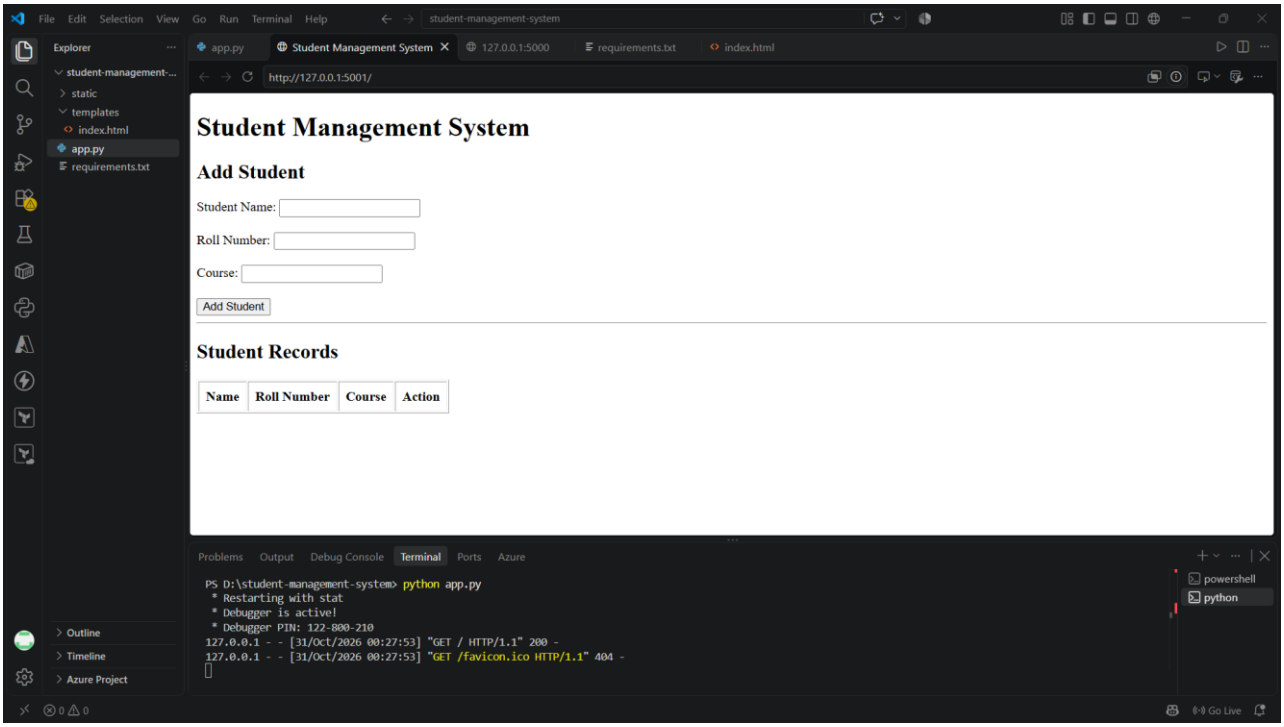



*Figure 1: Student Management System running locally during application verification.*

# 5. Project Directory Structure

```
student-management-system/
├── app.py
├── requirements.txt
├── Dockerfile
├── .dockerignore
├── .gitignore
├── templates/
│   └── index.html
└── k8s/
    ├── deployment.yaml
    └── service.yaml
```

app.py contains the Flask routes and startup configuration. templates/index.html defines the web interface. requirements.txt lists the Python dependency. Dockerfile defines the image build, while the k8s directory contains the Kubernetes Deployment and Service manifests.

# 6. Implementation and Execution

## Step 1: Verify Docker Installation

Docker was installed through Docker Desktop. The Docker CLI and engine were checked from PowerShell before building the application image.

```
docker --version
docker info
```

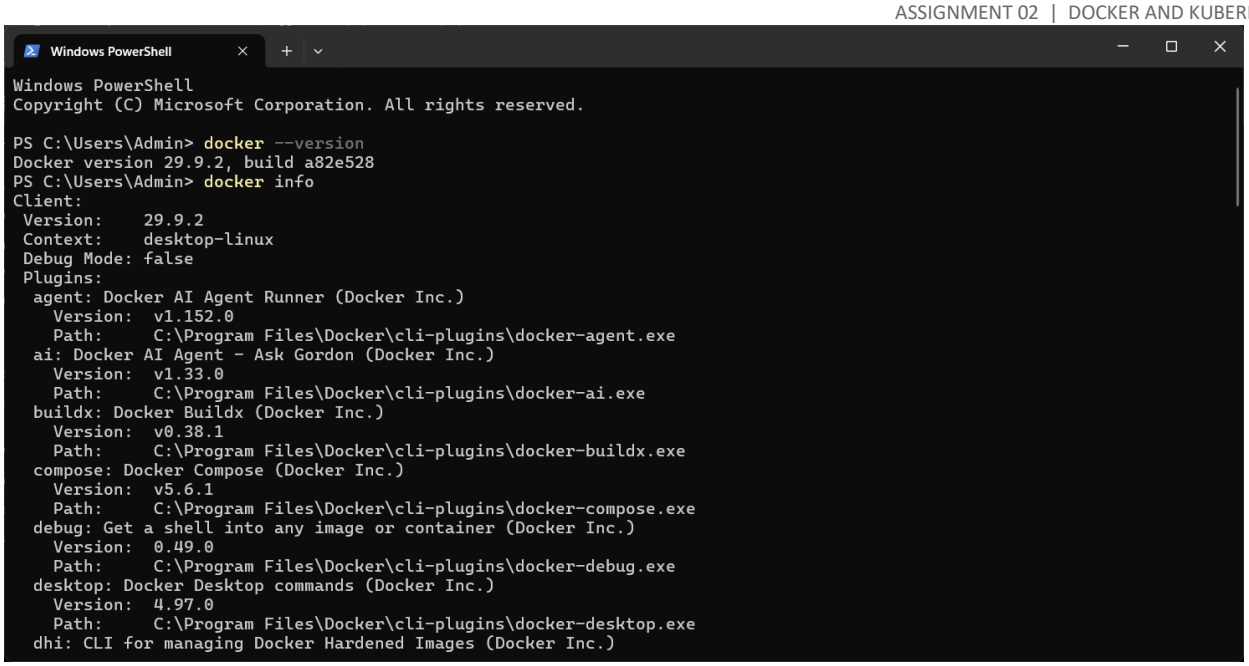



*Figure 2: Docker CLI version and Docker engine information verified in PowerShell.*

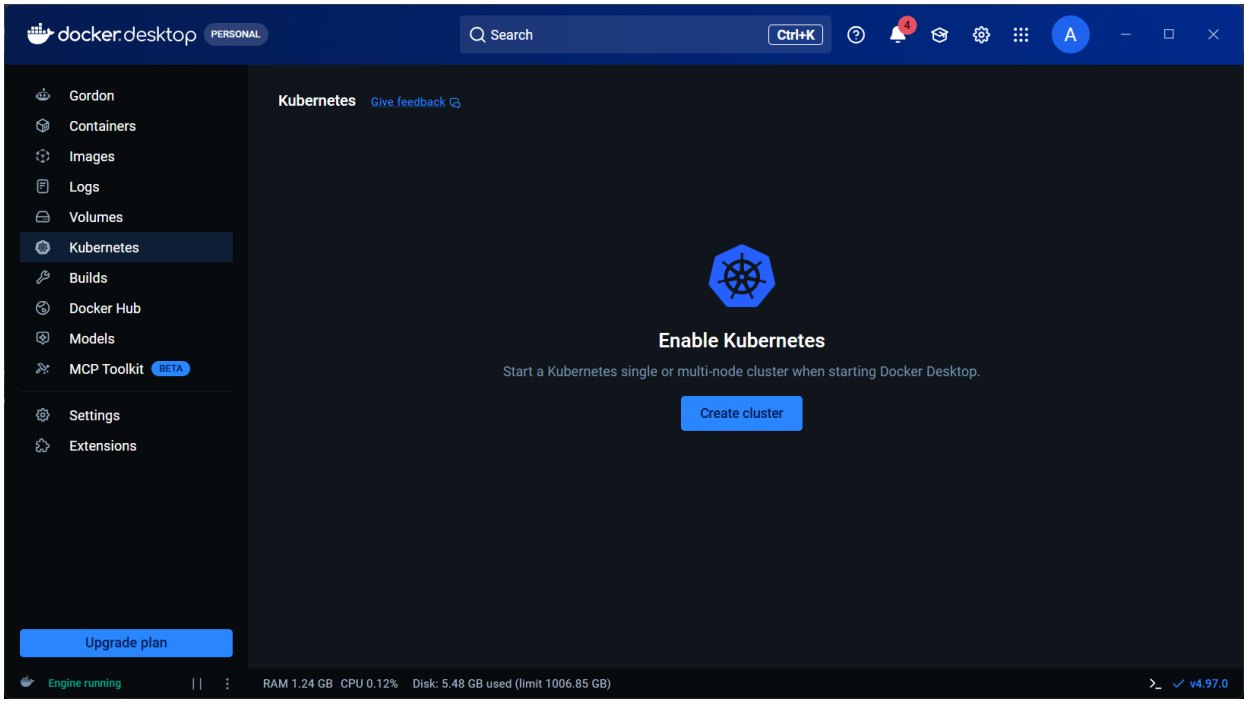



*Figure 3: Docker Desktop showing the local Kubernetes environment enabled.*

## Step 2: Prepare the Flask Application and Dependencies

The Flask application source and HTML template were organized in the project directory. The dependency file pins Flask to version 3.1.2.

```
Flask==3.1.2
```

The Flask application is started on all network interfaces at port 5000. This is required so traffic forwarded from Docker and Kubernetes can reach the web server.

```
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

## Step 3: Test the Application Locally

Before containerization, the application was started from the project directory and opened in a browser. The page displayed the Student Management System heading, Add Student form, and Student Records table.

```
python app.py
```

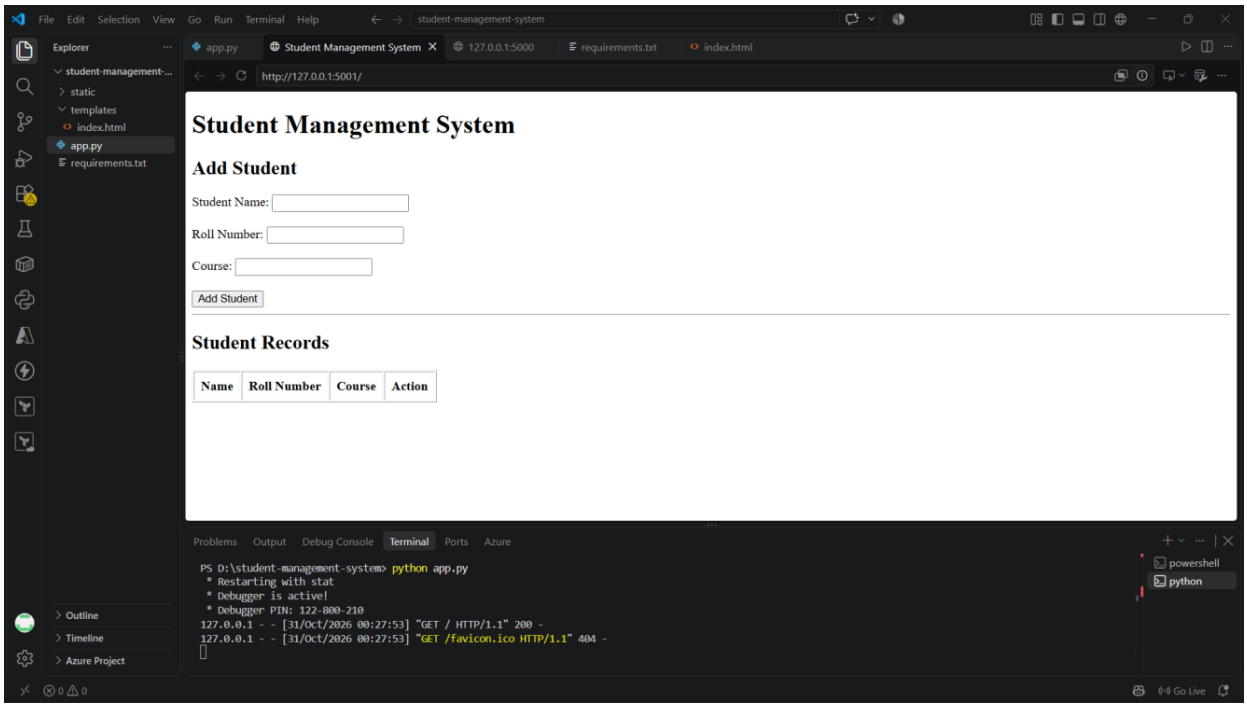



*Figure 4: Local Flask application displayed with the student entry form and records table.*

## Step 4: Create the Dockerfile

A Dockerfile was created to define how the application image is assembled. It uses a lightweight Python 3.11 base image, installs dependencies, copies the source code, exposes port 5000, and starts app.py.

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

The .dockerignore file excludes unnecessary local files from the build context.

```
__pycache__/
*.pyc
.venv/
venv/
.git/
.gitignore
```

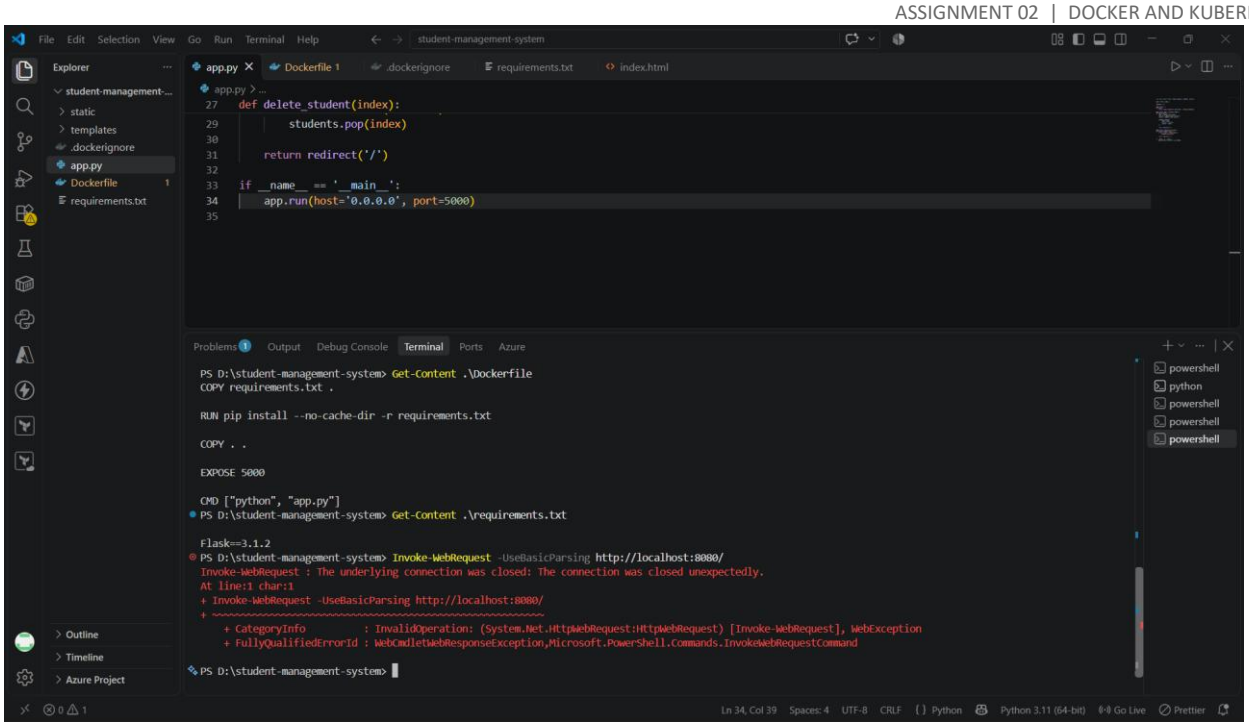



*Figure 5: Dockerfile and requirements file inspected in the VS Code terminal.*

## Step 5: Build the Docker Image

The image was built from the project root directory and tagged as student-management-system:1.0. The local image list was then checked to confirm that the image was available.

```
docker build -t student-management-system:1.0 .
docker images
```

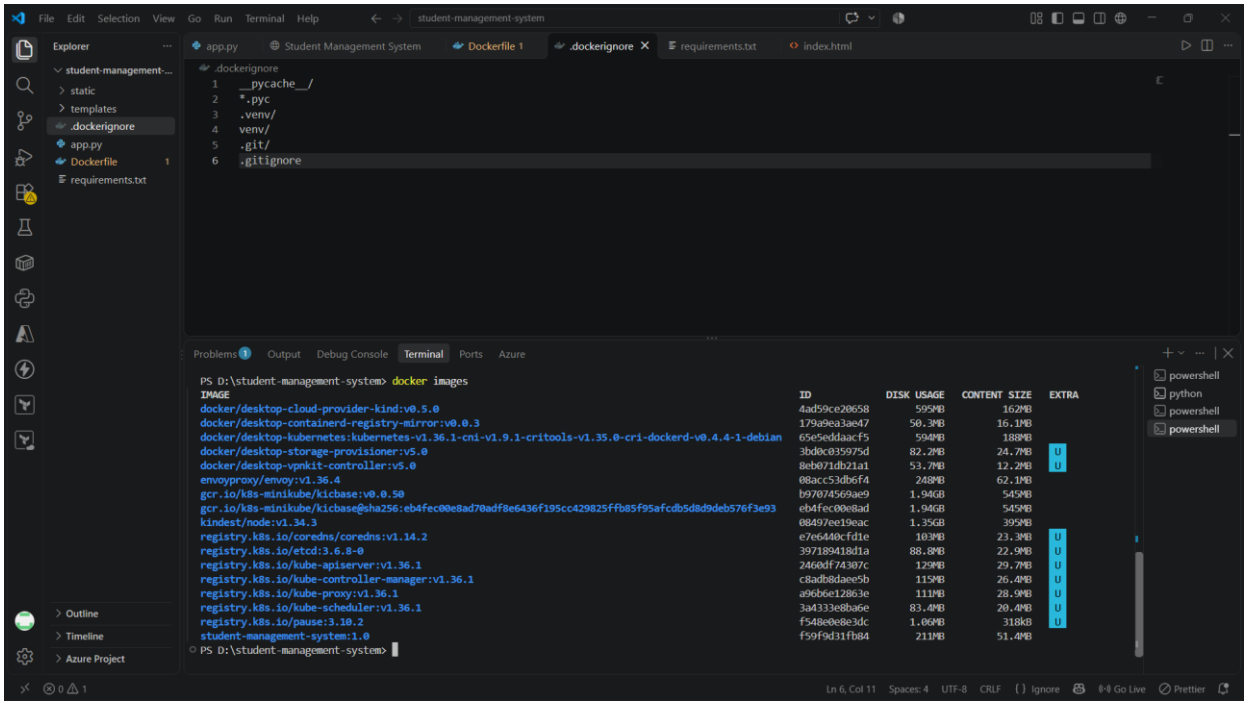



*Figure 6: Docker image list showing student-management-system:1.0.*

## Step 6: Run and Verify the Docker Container

The application image was started as a container with host port 8080 mapped to container port 5000. The container status and port mapping were checked using docker ps.

```
docker run -d --name student-management-app -p 8080:5000 student-management-system:1.0
docker ps
```

The container was shown in the Up state with the port mapping 0.0.0.0:8080->5000/tcp. The Docker-hosted application can be tested at http://localhost:8080.

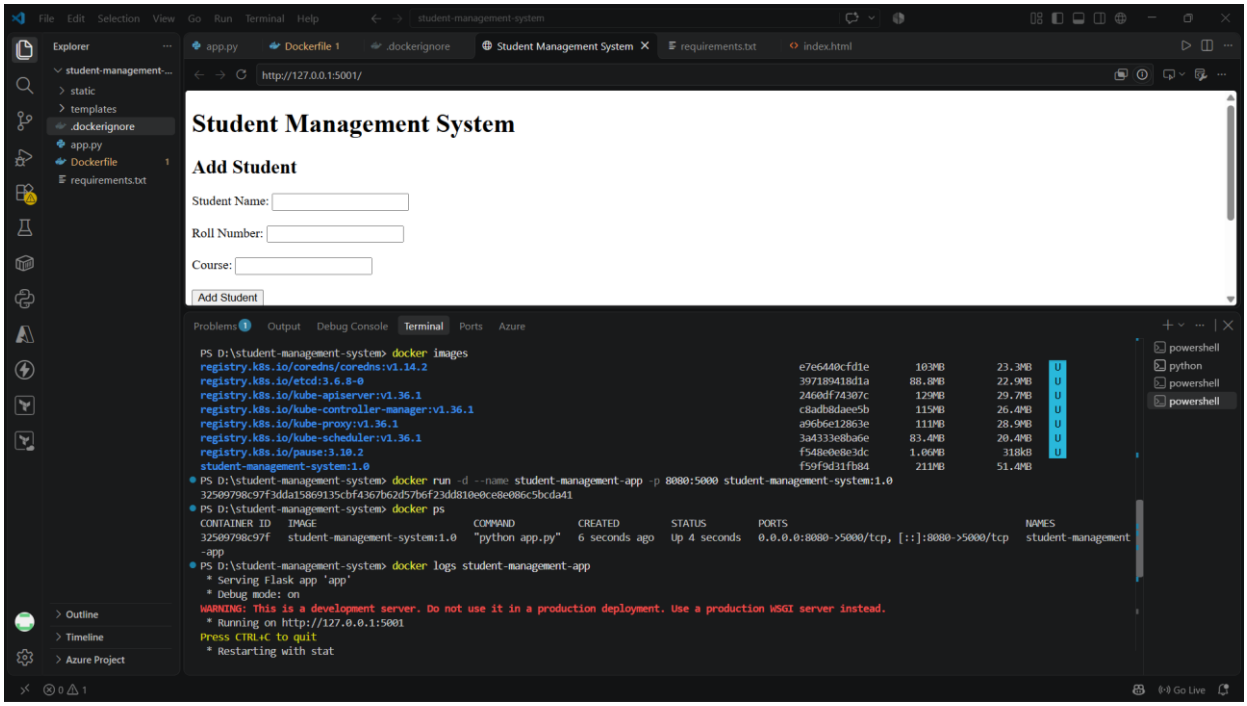



*Figure 7: Docker container running with host port 8080 mapped to container port 5000.*

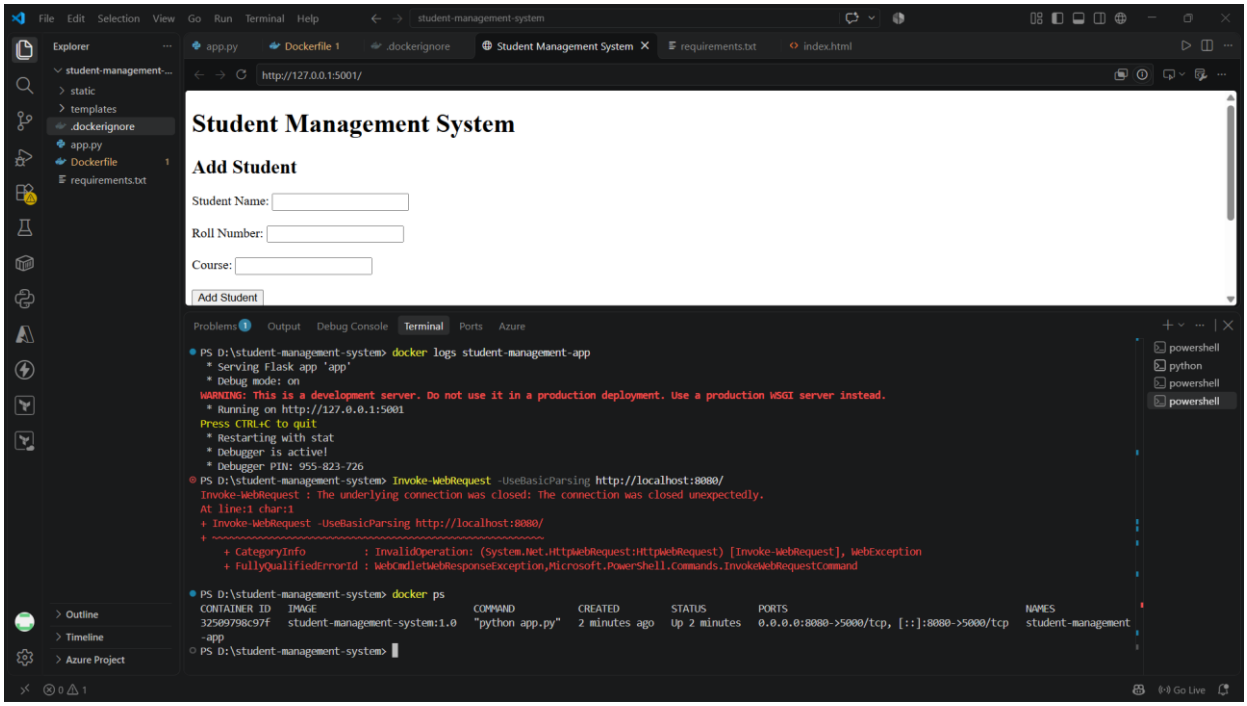



*Figure 8: Container status and localhost:8080 endpoint shown in the environment.*

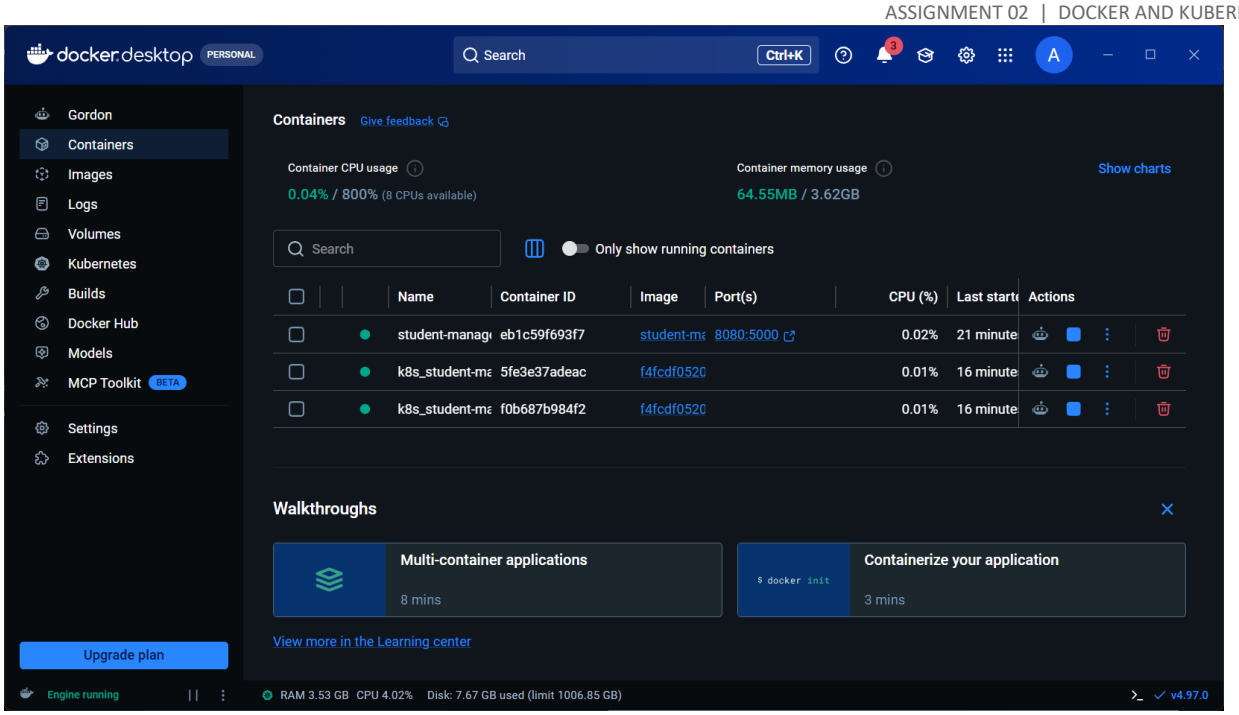



*Figure 15: Docker Desktop showing the application container and Kubernetes-managed containers running.*

# 7. Kubernetes Deployment

After verifying the Docker image and container, Kubernetes manifests were created in the k8s directory. The Deployment manages the required Pod replicas, and the Service provides a stable network endpoint.

## Step 7: Create the Kubernetes Deployment Manifest

The deployment.yaml manifest defines a Deployment named student-management-deployment. It specifies two replicas, uses the label app: student-management for selection, and runs the image student-management-system:1.0. The container exposes port 5000. The imagePullPolicy is IfNotPresent, which allows Kubernetes to use the image already available on the local node.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: student-management-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: student-management
  template:
    metadata:
      labels:
        app: student-management
    spec:
      containers:
        - name: student-management-container
          image: student-management-system:1.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 5000
```

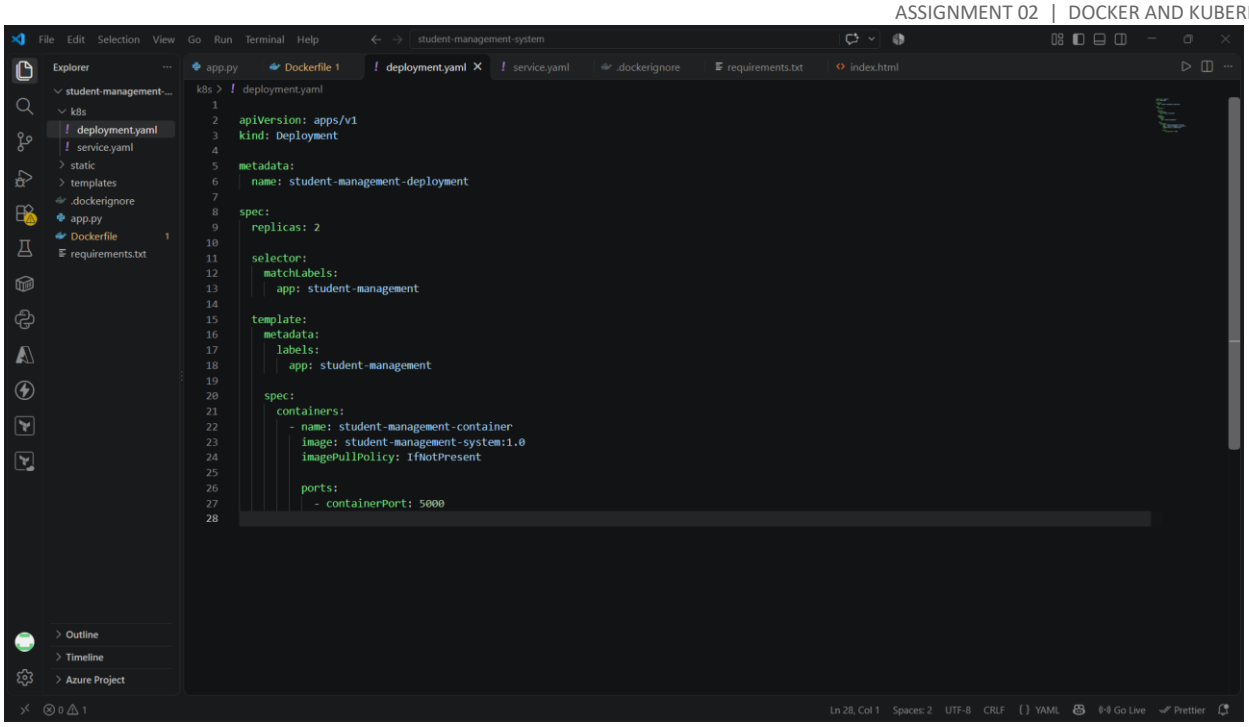



*Figure 9: Kubernetes Deployment manifest showing two replicas and the application container configuration.*

## Step 8: Create the Kubernetes Service Manifest

The service.yaml manifest defines a NodePort Service named student-management-service. It selects Pods with the label app: student-management, forwards Service port 5000 to targetPort 5000, and exposes NodePort 30080 for browser access.

```
apiVersion: v1
kind: Service
metadata:
 name: student-management-service
spec:
 type: NodePort
 selector:
  app: student-management
 ports:
  - port: 5000
    targetPort: 5000
    nodePort: 30080
```

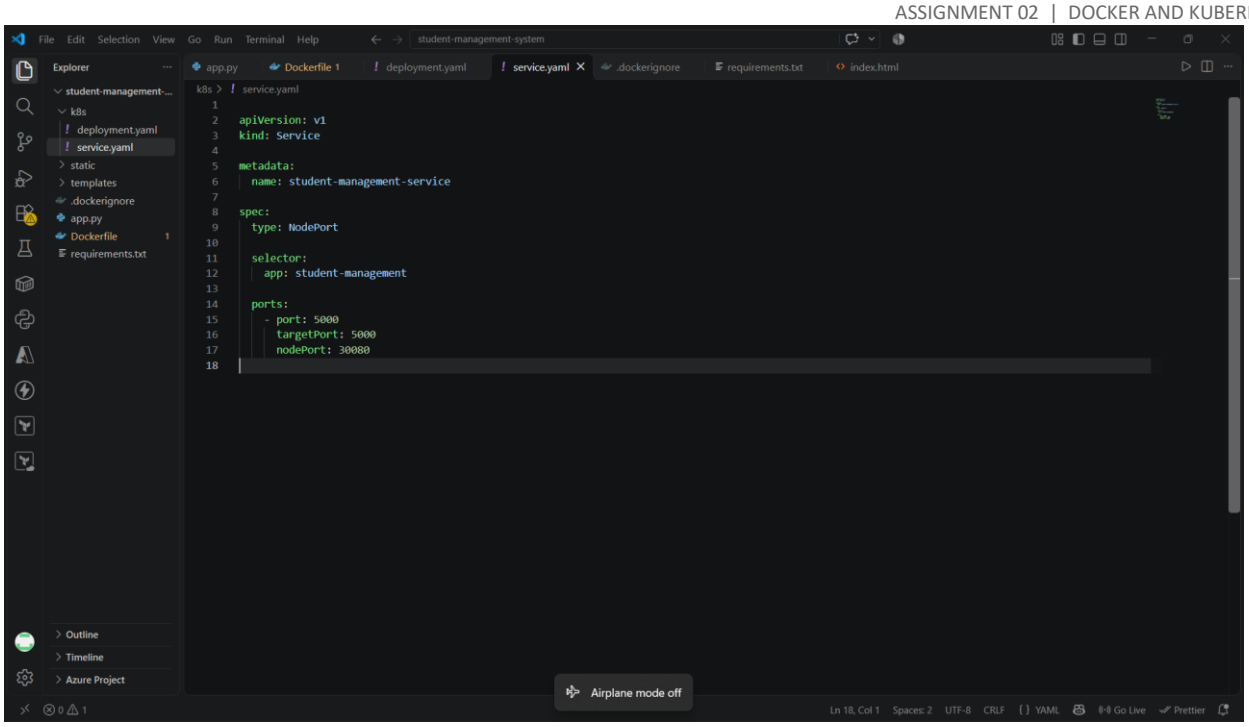



*Figure 10: Kubernetes Service manifest configured with NodePort 30080.*

## Step 9: Apply the Kubernetes Manifests

Both manifests were applied from the project root directory. Kubernetes returned confirmation that the Deployment and Service resources were created.

```
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
```

## Step 10: Verify Deployment, Pods, and Service

The Kubernetes resources were inspected using kubectl. The Deployment showed 2/2 replicas ready, up-to-date, and available. The two Pods each showed READY 1/1 and STATUS Running, with zero restarts. The Service listing showed NodePort mapping 5000:30080/TCP.

```
kubectl get deployments
kubectl get pods
kubectl get services
```

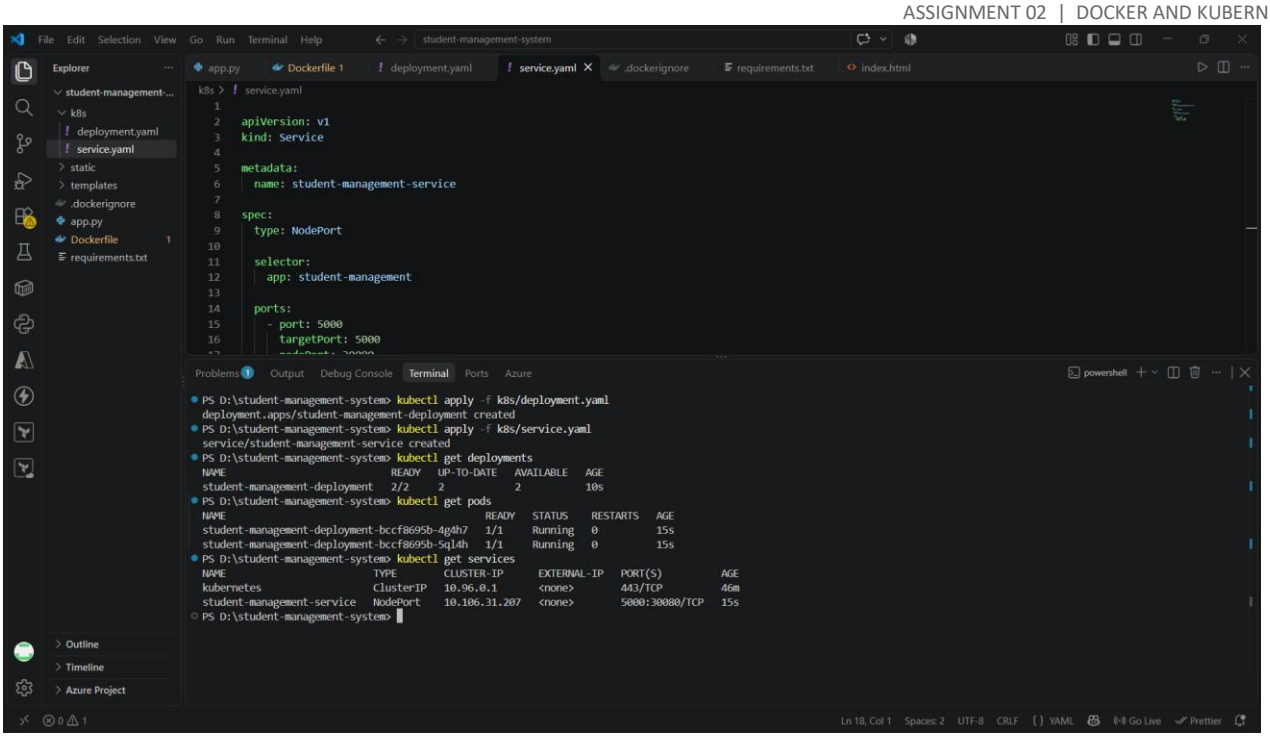



*Figure 11: Successful resource creation; Deployment 2/2 available, both Pods Running, and NodePort Service 5000:30080/TCP.*

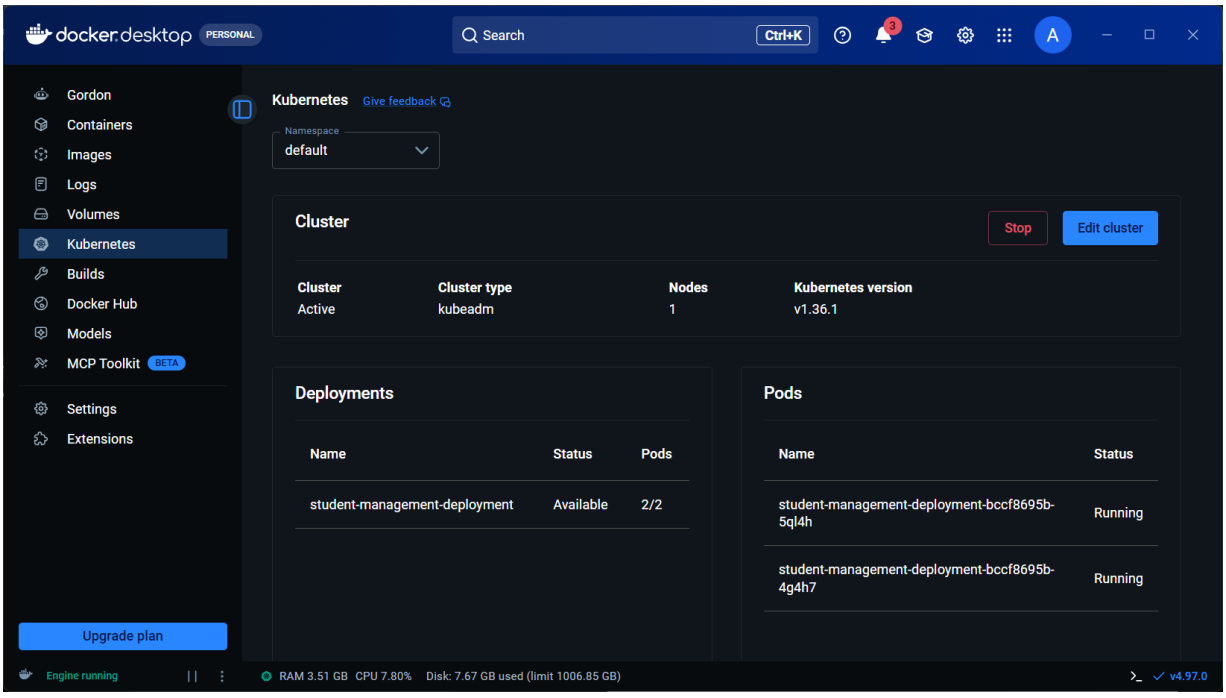



*Figure 16: Docker Desktop Kubernetes dashboard showing an active cluster, an available Deployment with 2/2 Pods, and both Pods in Running state.*

## Step 11: Access the Application through NodePort

The application was opened in a browser using the NodePort URL http://localhost:30080. The Student Management System page loaded successfully, confirming that browser traffic reached the application through the Kubernetes Service.

```
http://localhost:30080
```

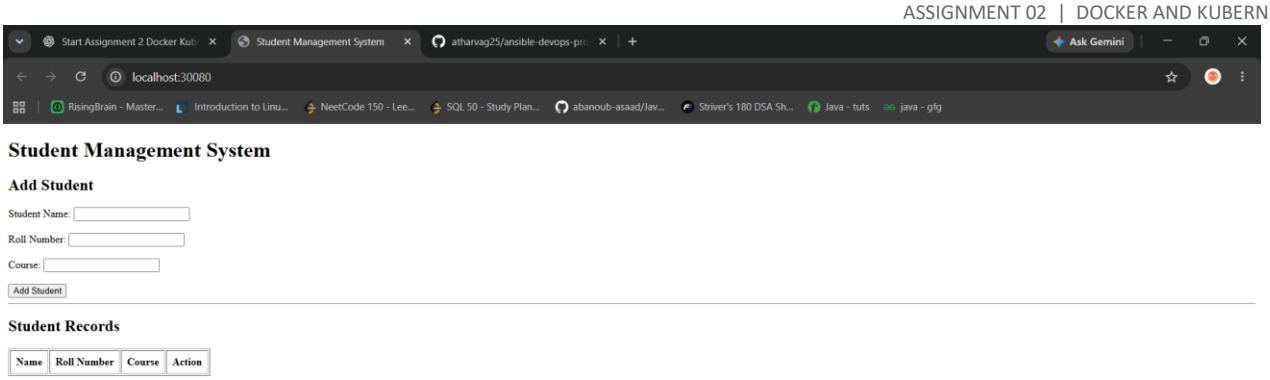



*Figure 12: Student Management System accessed through the Kubernetes NodePort endpoint.*

# 8. GitHub Version Control and Repository

The project files were staged and committed using Git. The project was published to the public GitHub repository named flask-app-docker-k8s. The repository name reflects the technologies used in the assignment.

```
git add .
git commit -m "Initial commit: Student Management System with Docker and Kubernetes"
git remote add origin https://github.com/atharvag25/flask-app-docker-k8s.git
git push -u origin main
```

If the origin remote is already configured, the git remote add origin command should be skipped. The repository can be viewed at:

**https://github.com/atharvag25/flask-app-docker-k8s**

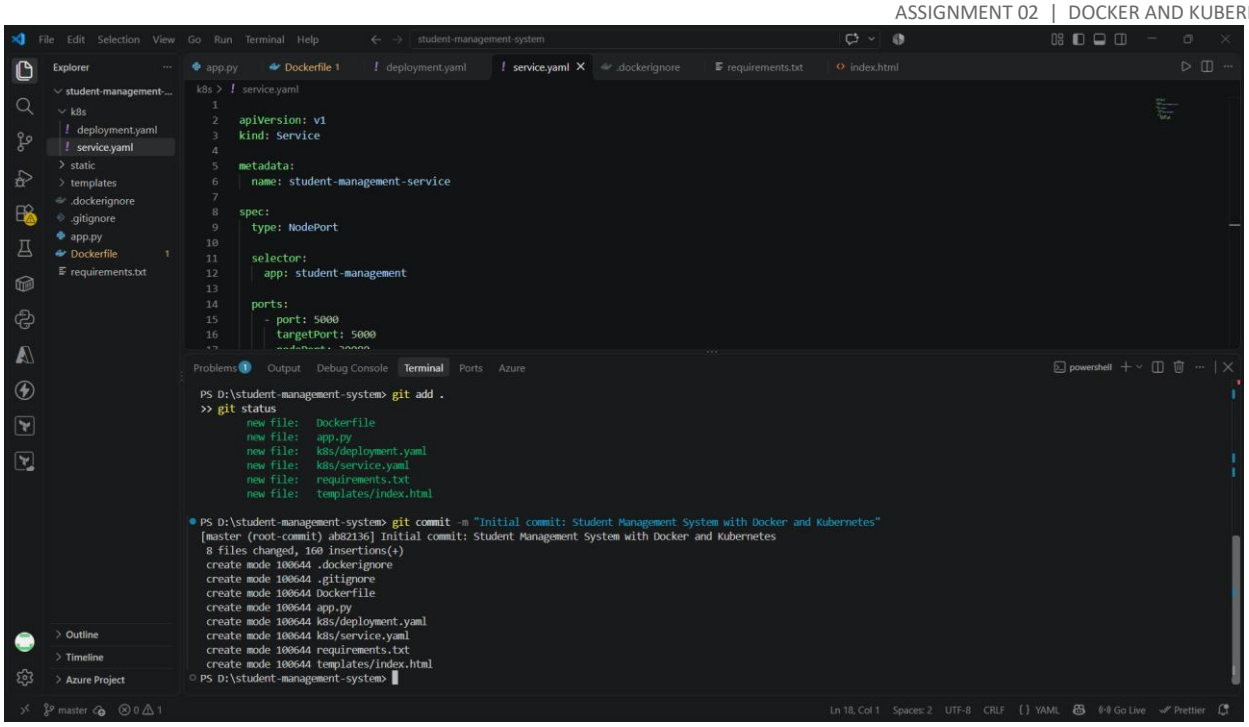



*Figure 13: Git staging and initial commit containing the application and Kubernetes files.*

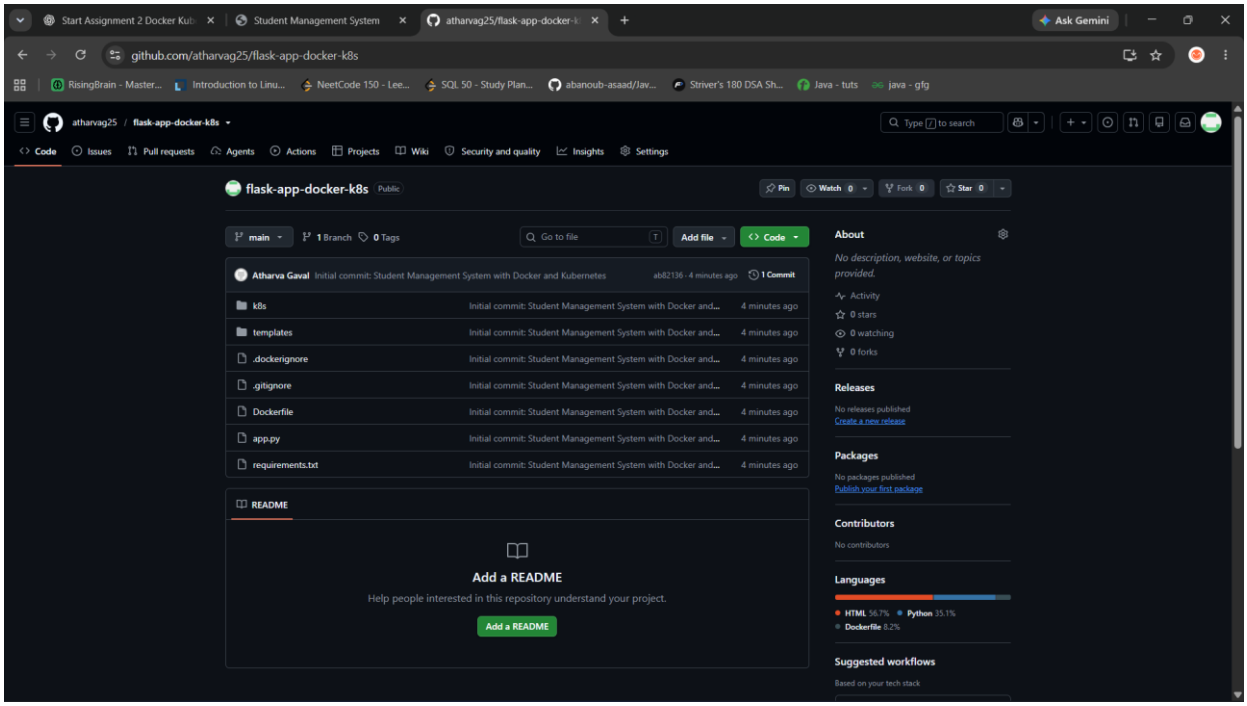



*Figure 14: GitHub repository flask-app-docker-k8s showing the published project files.*

# 9. Result

The Flask Student Management System was successfully containerized using Docker and deployed on the local Kubernetes cluster. The Kubernetes Deployment maintained two running replicas, and the NodePort Service exposed the application on port 30080. The application was accessible through http://localhost:30080, and the source code was published to GitHub.

# 10. Conclusion

This assignment demonstrated the end-to-end workflow of containerizing a Python web application and deploying it using Kubernetes. Docker provided a consistent application runtime, while Kubernetes Deployment and Service resources provided replica management and network exposure. The successful verification of both Pods, the NodePort mapping, and browser access confirmed that the deployment was functioning as intended.

# 11. Learning Outcomes

- Understood Docker images, containers, Dockerfiles, and build context.
- Learned to package a Flask application with its Python dependencies.
- Practiced running containers and mapping host ports to container ports.
- Understood Kubernetes Deployments, Pods, replicas, labels, and selectors.
- Configured a NodePort Service to expose an application externally.
- Used kubectl to apply and inspect Kubernetes resources.
- Published the project using Git and GitHub.